Chapter 8: Integrating Your Application with the Windows Ecosystem

Windows isn't just a platform that enables you to run your applications, it offers a wide range of features to increase your productivity, such as with **Windows Hello**, which you can use to log in in a safe and seamless way thanks to biometric systems such as face recognition and fingerprint readers, **location services**, to identify the position of the user, and **Sharing**, which you can use to easily transfer information from one application to another.

Windows offers a developer ecosystem that enables you to light up all these features directly in your applications to make them even more powerful. For example, the way that you can log in to your PC just by using your camera thanks to Windows Hello, you can enable that same experience in your applications. At the same time, since we're building a desktop application, it means that we can integrate with the operating system in ways that aren't supported by other types of applications, such as working with the filesystem, integrating web experiences, and much more.

In this chapter, we're going to explore a few of these techniques to integrate your application with the various features that Windows makes available to developers, such as the following:

- Integrating APIs from the Universal Windows Platform
- Working with files and folders

- Supporting the sharing contract
- Integrating web experiences into your desktop application

Let's start!

Technical requirements

The code for the chapter can be found here:

https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter08

Integrating APIs from the Universal Windows Platform

Since the introduction of Windows 10, the Windows team has deeply invested in the **Universal Windows Platform** (**UWP**). Consequently, UWP became the entry point for all developers to enable any new feature added to Windows. However, in the past, this meant that only by building UWP apps were you able to integrate new features such as Windows Hello, geolocation, the sharing contract, and so on. As we discussed in ***Chapter 1***, *Getting Started with the Windows App SDK and WinUI*, this approach led to slow adoption of the platform, especially by enterprise developers, since most of the time it meant rewriting existing .NET applications almost from scratch.

In recent years, as such, the Windows team has worked on ways to enable the usage of the UWP ecosystem in existing .NET applications, leading developers to enhance their existing applications with new features, without the need to restart from scratch with new technology.

The most recent outcome of this effort is C#/WinRT, which is a library that enables C# applications to consume APIs that belong to Windows Runtime, which is the framework the UWP is based on. This library has been built with the same guiding principles as the Windows App SDK: lifting the C# projection for Windows Runtime from the operating system to a library, so that it can be evolved and supported independently from the operating system.

However, when it comes to .NET applications, the integration is even deeper than what we can achieve just with the Windows App SDK. You won't need, in fact, to manually install any NuGet package, since this integration is included in the specific .NET target framework for Windows. You have already seen this configuration in action when we looked at the project file of a WinUI application. If you remember, this is how the **`TargetFramework`** property is set when you create a new WinUI project using the dedicated Visual Studio template:

```
<TargetFramework>net6.0-
windows10.0.19041.0</TargetFramework>
```

This property can have different values, based on the target SDK you want to leverage in your application:

- **`net6.0-windows10.0.17763.0`**
- **`net6.0-windows10.0.18362.0`**
- **`net6.0-windows10.0.19041.0`**
- **`net6.0-windows10.0.22000.0`**

It's important to highlight how the target Windows SDK version doesn't mean that the application will run only on that specific version of Windows, but that the application will be compiled against that specific SDK, enabling you to use a different set of APIs. The UWP, in fact, evolved over time and each SDK released included new features and APIs you can use in your applications. This means that, for example, an application that targets the 10.0.22000.0 SDK (which matches the Windows 11 release) can run without issues also on Windows 10, as

long as you don't use specific Windows 11 features. This approach is empowered by the capability detection feature of the UWP, which you can use to detect if a specific feature is available before using it and, eventually, fall back to a different approach.

Since this implementation is based on the .NET target framework, the usage of UWP features isn't directly connected to the Windows App SDK. To use them, in fact, you aren't required to install the Windows App SDK NuGet package, you just have to switch your current target framework to one of the specific Windows 10/11 ones.

If you're using WinUI as a UI framework to build your application, you're already set up. As we saw in ***Chapter 1***, *Getting Started with the Windows App SDK and WinUI*, the **`TargetFramework`** property of any new WinUI project is already set in the right way. If you have an existing WPF or Windows Forms project based on .NET 5 or .NET 6, instead, it's very likely that your **`TargetFramework`** will look like this:

```
  <TargetFramework>net6.0-windows</TargetFramework>
```

As you will notice, this target doesn't explicitly declare a specific version of Windows. This means that you will indeed be able to use specific Windows APIs, but that belong to the broader Win32 ecosystem, such as access to the Windows Registry, communication with Windows services, and so on. With this target, your application will also run on older versions of Windows, such as Windows 8 or Windows 7. However, you won't have access to any of the new features introduced in Windows 10. As such, the first step to integrate new APIs is to change the current **`TargetFramework`** to one of the specific Windows 10 ones, as outlined before.

Regardless of the UI platform of your choice, once you switch to a specific Windows 10 or 11 target framework, the C#/WinRT library will be automatically installed in your project. You can see that by expanding the **Dependencies** section of your application:

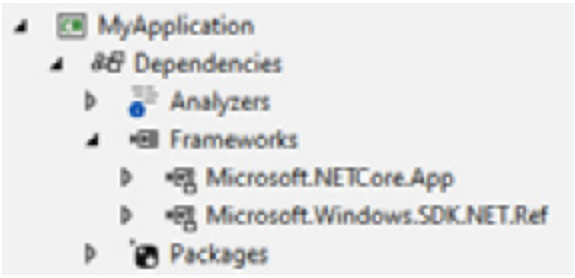

Figure 8.1 – The C#/WinRT library included in a .NET application

Inside the **Frameworks** node, other than the basic .NET 6 runtime (identified by the **`Microsoft.NETCore.App`** framework), we can see also a dependency called **`Microsoft.Windows.SDK.NET.Ref`**. This is the projection that enables us to use APIs that are part of the Windows Runtime inside our .NET application.

Now that we have learned how we can integrate Windows APIs into our .NET application (independently from the UI framework), let's see a couple of real examples.

Getting the location of the user

Windows includes a powerful set of APIs that you can use to get the location of the user, which makes it a precious companion for many consumer and enterprise scenarios: displaying the position of the user on a map in a delivery app; getting their location when they complete a task in a to-do application for first-line workers, and many more.

The heart of this feature is the **`Geolocator`** class, which belongs to the **`Windows.Devices.Geolocation`** namespace. Let's see a brief example of how to use it:

```
private async Task GetCurrentPositionAsync()
{
    Geolocator = new Geolocator();
    Geoposition position = await geolocator.
        GetGeopositionAsync();
```

```
    Console.WriteLine($"
{position.Coordinate.Point.Position.
        Latitude} -
{position.Coordinate.Point.Position.
          Longitude}");
}
```

The **`Geolocator`** class exposes an asynchronous method called **`GetGeopositionAsync()`**, which returns a **`Geoposition`** object that includes a lot of information about the current position of the user. The most important one is the **`Coordinate`** property: through the **`Point.Position`** object, you can access the latitude and longitude coordinates, as in the previous example.

This code, however, has a flaw. Windows gives full control to the user around privacy and security. As such, users have the option, through the Settings app, to disable the location services entirely or for a specific application. This is one of the areas where there's a significant difference compared to when you use the same APIs in the UWP application. The UWP, in fact, due to the sandbox in which applications run, has a granular capability model, which allows you to opt in when you want to use Windows features that have special implications around privacy and security. As such, UWP apps that use location services are not enabled by default and you must do the following:

1. Declare the location capability in the manifest.
2. Call the **`RequestAccessAsync()`** method exposed by the **`Geolocator`** class as the first step, which will trigger a popup asking for the user's permission to use the location services.

Win32 applications aren't able to leverage the same capabilities model but they can access all the features exposed by Windows. As such, .NET applications that use the location APIs have the opposite behavior: by default, they are granted access to the location services. The user has the following different options to disable them:

- Completely disable the location services in Windows, blocking every application from retrieving the position (option **1** in the following screenshot).
- If the application is deployed as packaged, it will show up in the list of apps available in the **Privacy & security** | **Location** section of the **Settings** app (option **2** in the following screenshot, which shows an example of a packaged .NET application called **MyApplication**).
- If the application is deployed as unpackaged, instead, it doesn't have an identity. As such, Windows isn't able to manage the permissions in a granular way. The application will show up in the generic list of desktop apps and, through a toggle, you can enable or disable access to all of them (option **3** in the following screenshot shows a list of unpackaged apps that have used the location services).

This is the screenshot showing an example of the three scenarios:

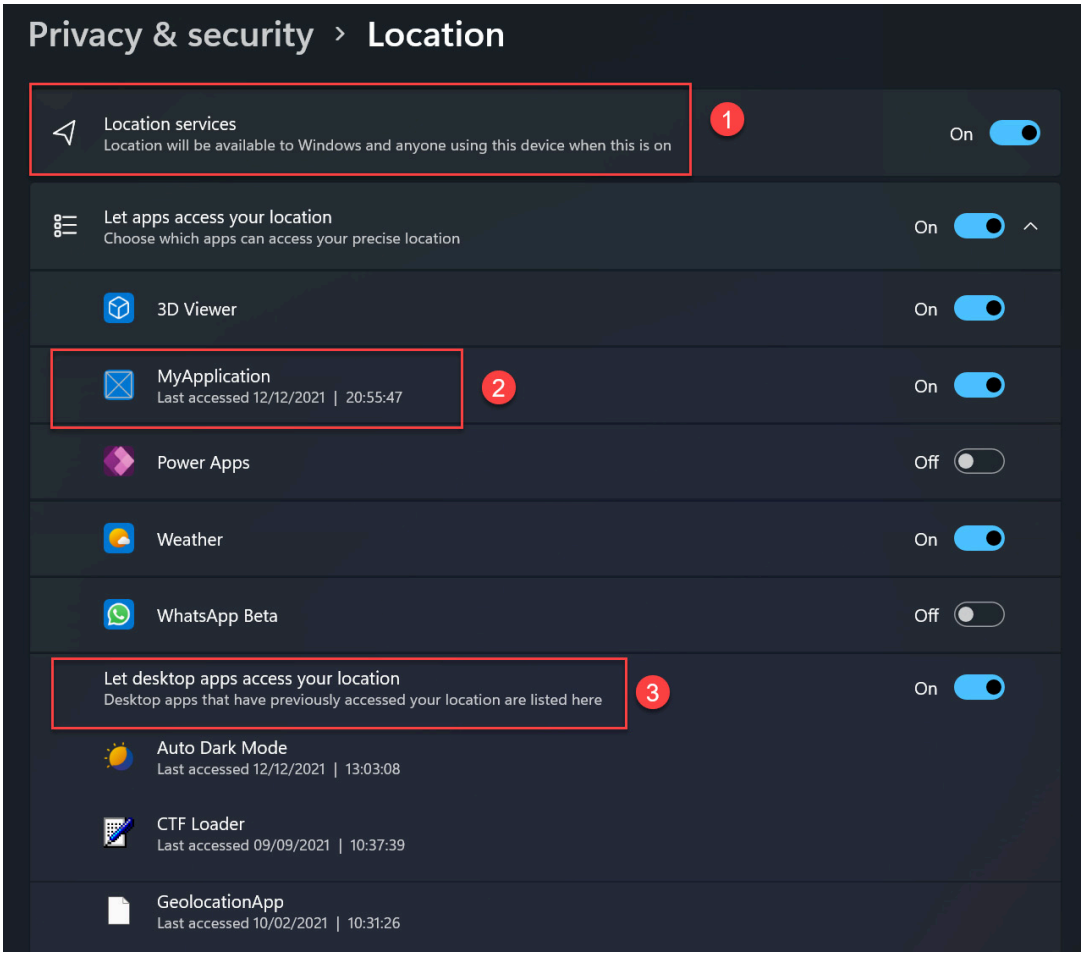

Figure 8.2 – The location services configuration included in Windows 11

Now that we know how Windows manages the security and privacy of location services, we can look at a better way to use the Geolocation APIs:

```
private async void OnGetPosition(object sender,
  RoutedEventArgs e)
{
    Geolocator geolocator = new Geolocator();
    if (geolocator.LocationStatus !=
PositionStatus.Disabled &&
      geolocator.LocationStatus !=
PositionStatus.NotAvailable)
    {
        Geoposition position = await geolocator.
          GetGeopositionAsync();
        Console.WriteLine($"{position.Coordinate.Point.
          Position.Latitude} - {position.Coordinate.
            Point.Position.Longitude}");
    }
}
```

The status of the location services is stored inside the **`LocationStatus`** of the **`Geolocator`** property, through the **`PositionStatus`** enumerator. We get the position of the user only if this status is different from **`Disabled`** or **`NotAvailable`**. It's very important to include this check, otherwise, you will get an exception if you try to invoke the **`GetGeopositionAsync()`** method when the location services are disabled.

The **`Geolocator`** class also supports the option to continuously detect changes in the position of the user, as GPS navigation apps do. The following code shows this feature in action:

```
private async void OnGetPositionChanges(object sender,
  RoutedEventArgs e)
{
    Geolocator geolocator = new Geolocator();
    if (geolocator.LocationStatus !=
PositionStatus.Disabled &&
      geolocator.LocationStatus !=
PositionStatus.NotAvailable)
    {
        Console.WriteLine(geolocator.LocationStatus);
        geolocator.PositionChanged += Geolocator_
          PositionChanged;
    }
}
private void Geolocator_PositionChanged(Geolocator
sender,
  PositionChangedEventArgs args)
{
    Console.WriteLine($"
{args.Position.Coordinate.Latitude} -
      {args.Position.Coordinate.Longitude}");
}
```

All you have to do is to subscribe to the **`PositionChanged`** event, which, in the event arguments, gives you a **`Position`** object with all the information about the location change that has just been detected.

Now we know how to use the Geolocator API and how to retrieve the latitude and longitude of the current position. What can we do with this information? Let's see what we can achieve thanks to the Bing Maps service.

Taking advantage of the Bing Maps service

Bing Maps isn't just a mapping service for the consumer, it's also a set of APIs and controls that you can use in your applications to enhance them. Some of these APIs are built inside the UWP, so you can use them without having to manually perform network operations and parse JSON payloads. In this section, we're going to see how we can use these services to turn our coordinates into a human-readable address.

The first step is to register an application on the Bing Maps portal, which is available at **https://www.bingmapsportal.com/**.

After you have logged in with your Microsoft account, click on the label **Click here to create a new key** to access the registration form. You will be asked the following questions – make sure to set the fields in the same way as in the following screenshot:

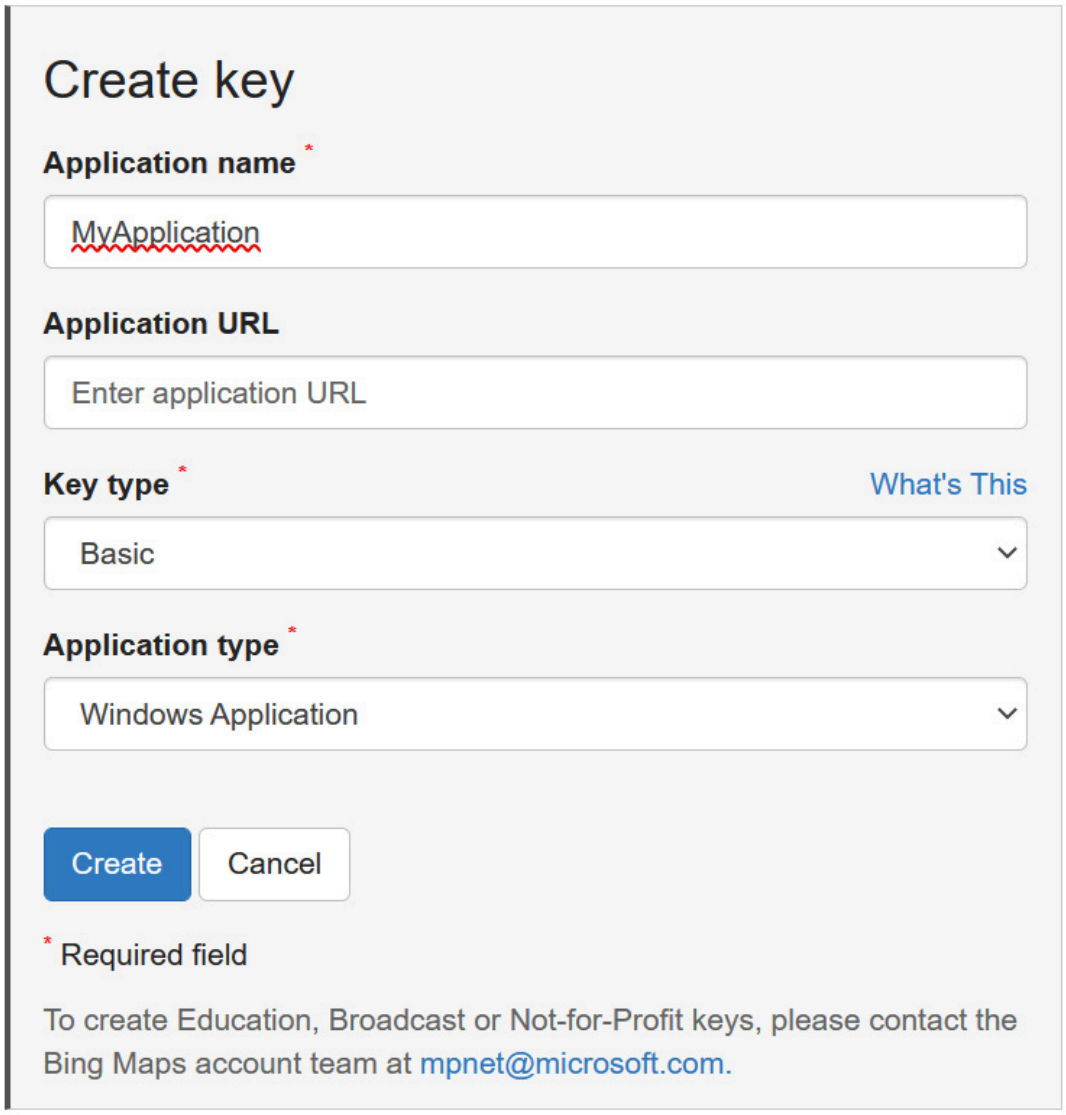

Figure 8.3 – The form to register a new Bing Maps key

After you have clicked the **Create** button, the key will show up in the following dashboard:

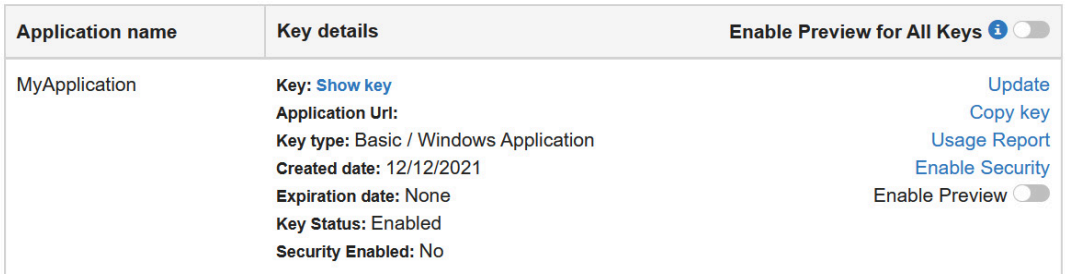

Figure 8.4 – A Bing Maps key in the dashboard

For security reasons, by default, the key won't be visible. You can either click on the **Show key** button and copy it, or directly click on the

Copy key button. Either way, once the key is stored on your clipboard, you can go back to your application and set it inside the **`ServiceToken`** property of the **`MapService`** class (which belongs to the **`Windows.Services.Maps`** namespace), as in the following example:

```
public MainPage()
{
    this.InitializeComponent();
    MapService.ServiceToken = "<your key>";
}
```

Now, thanks to this key, you'll be able to use many other APIs included in the **`Windows.Services.Maps`** namespace. Let's see the usage of the **`MapLocationFinder`** class, which you can use to perform geocode (to retrieve the coordinates starting from an address) and reserve geocode (to retrieve the address starting from the coordinates) operations. In our scenario, thanks to the **`Geolocator`** class, we already have a set of coordinates, so we're going to perform reverse geocoding:

```
private async void myButton_Click(object sender,
  RoutedEventArgs e)
{
    Geolocator geolocator = new Geolocator();
    if (geolocator.LocationStatus !=
      PositionStatus.Disabled &&
geolocator.LocationStatus
        != PositionStatus.NotAvailable)
    {
        Geoposition position = await
          geolocator.GetGeopositionAsync();
        MapLocationFinderResult result =
    await
MapLocationFinder.FindLocationsAtAsync(position.
      Coordinate.Point);
        Console.WriteLine(result.Locations[0].DisplayNa
me);
```

```
        }
    }
```

The **`Geolocator`** class is a great companion for the **`MapLocationFinder`** one since they use the same types to manage location data. As such, to perform reverse geocoding, it's enough to call the **`MapLocationFinder.FindLocationsAtAsync()`** method passing, as a parameter, the **`Coordinate.Point`** property of the **`Geoposition`** object returned by the **`Geolocator`** class.

When you do reverse geocoding, the result contains a property called **`Locations`**: it's a collection since the APIs can return multiple places for the same coordinates. In this example, we're showing the information about the first result. The **`DisplayName`** property contains a human-readable version of the full address, but you can also access more granular properties through the **`Address`** property, such as city, postcode, region, and so on.

NOTE

The UWP includes a control called **`MapControl`**, *which is a great companion of the location APIs since you can use it to display the location of the user on a map, show the points of interest, calculate a path, and so on. Unfortunately, at the time of writing,* **`MapControl`** *hasn't been ported to WinUI and the Windows App SDK. It will be included in a future release.*

Let's see another example of the integration of APIs from the UWP: Windows Hello.

Introducing biometric authentication

Windows includes a security platform called Microsoft Passport, which provides a safe way to store sensitive information such as your account credentials. One of the most interesting features of Microsoft Passport is Windows Hello, which is able to generate a token to sign in

based on biometric parameters, such as face recognition or fingerprint readings. Windows Hello makes logging in to your computer safer in the following ways:

- It removes the possibility of credential theft. Since authentication happens with a two-factor authentication system based on a biometric component or on a PIN, you remove the requirement of using a password, which is usually the weak link in the security chain.
- The authentication system is based on a component that is only local and stored in a safe way, thanks to the usage of the **TPM** (short for **Trusted Platform Module**) chip, which enables hardware protection. Being only local means that the attacker would need physical access to your computer to steal your data, while with a regular password an attacker is normally able to access your data even remotely.

To work properly, Windows Hello requires your Windows account to be connected to a Microsoft account (in a consumer scenario) or to an Azure Active Directory account (in an enterprise scenario). Either way, when you set up Windows Hello, the operating system generates a public-private key pair on the device; the private key is generated and stored by the TPM, making it virtually impossible to access it. The keys are bound to a specific user so, if you have multiple users on the same machine, each of them will need to set up Windows Hello and the TPM still stores their own public-private key pair.

Windows Hello, from a developer's point of view, can be used for many advanced scenarios. For example, you can create a specific application key for the user, which gives you access to information such as the public key or the attestation. This way, you can link this data to a user profile, enabling you to use Windows Hello as an authentication system also for server-side scenarios (such as a backend API).

For the purpose of this book and to show you the integration with the Windows ecosystem, instead, we'll just focus on the simplest scenario: logging the user into an application. In this scenario, you typically authenticate the user first with a traditional system (such as a username and a password) and then you give them the option, after the first successful login, to enable Windows Hello to avoid inserting the credentials each time. In this scenario, you don't need to store any information on the backend, since you're using Windows Hello only as a local authentication system. If the same user needs to log in on another machine, they will need to repeat the process starting with their credentials.

This scenario is easy to implement, thanks to the APIs that are included in the **`Windows.Security.Credentials`** and **`Windows.Security.Credentials.UI`** namespaces:

```
private async Task AuthenticateUserAsync()
{
    bool keyCredentialAvailable = await
KeyCredentialManager.
      IsSupportedAsync();
    if (keyCredentialAvailable)
    {
        var result = await UserConsentVerifier.
          RequestVerificationAsync("Checking if it's
            really you");
        if (result ==
UserConsentVerificationResult.Verified)
        {
            //continue the operation
        }
        else
        {
            //access is denied
        }
```

```
        }
    }
```

As a first step, we must check if Windows Hello is enabled on the machine; otherwise, we need to fall back to a traditional authentication system. This information is returned by the **`IsSupportedAsync()`** method exposed by the **`KeyCredentialManager`** class, which returns a simple **`true`** / **`false`**.

If Windows Hello is available, we can proceed and trigger a user verification, by calling the **`RequestVerificationAsync()`** method of the **`UserConsentVerifier`** class. As a parameter, we pass a message that we want to display inside the authentication popup, as you can see in the following screenshot:

Figure 8.4 – The Windows Hello popup with a custom message

As an outcome, you will get a value of the **`UserConsentVerificationResult`** enumerator. If it's equal to **Verified**, it means that the authentication has been completed successfully, so we can move on with the operation we are protecting. Otherwise, we

block the execution since we haven't been able to properly authenticate the user.

Windows Hello can be used for more advanced scenarios, including server-side ones. The key class to support them is **`KeyCredentialManager`**, which we have already seen, which offers you methods such as **`RequestCreateAsync()`**, which you can use to generate a private-public key pair for the user. The following example shows this scenario:

```
private async Task AuthenticateUserAsync()
{
    bool keyCredentialAvailable = await
KeyCredentialManager.
      IsSupportedAsync();
    if (keyCredentialAvailable)
    {
        var keys = await KeyCredentialManager.
          RequestCreateAsync("username",
            KeyCredentialCreationOption.ReplaceExisting
);
        if (keys.Status == KeyCredentialStatus.Success)
  {
        var publicKey =
keys.Credential.RetrievePublicKey();
    var attestation = await keys.Credential.
      GetAttestationAsync();
}
    }
}
```

We pass to the **`RequestCreateAsync()`** method a unique identifier for the user (in the preceding code, it's a fixed string named **`username`**, but it could be the user email address or account identifier). If the operation completes successfully, we can use the **`Credential`** property of the

returned object to perform tasks such as retrieving the public key or the attestation, so that we can send it to our backend.

As mentioned at the beginning of the section, we aren't going to look at this scenario in detail in this book. If you want to know more, you can read the official documentation at **https://docs.microsoft.com/en-us/windows/uwp/security/microsoft-passport**, which also contains a step-by-step tutorial on how to build a client-server architecture based on Windows Hello.

Thanks to the integration with the UWP, we also have access to a new set of APIs to work with files and folders. Let's take a deeper look!

Working with files and folders

One of the most common requirements of a desktop application is to work with files and folders to support a wide range of scenarios: from writing and reading a file to accessing the content of a folder. .NET includes a namespace dedicated to this scenario, called **System.IO**, which includes many APIs to work with files, directories, and streams. However, we won't cover them in this chapter, because these APIs have been mostly unchanged over the years and, in this book, we're assuming you have a basic knowledge of the .NET ecosystem and Windows desktop development.

The UWP, instead, has introduced a new set of APIs to work with files and folders that are especially important if you have adopted WinUI as a platform. The classic .NET APIs will continue to work but there are some scenarios (such as enabling users to pick a file from a folder) that will require you to use these new APIs. Let's explore them!

Working with folders

Folders are represented by the **`StorageFolder`** class, which belongs to the **`Windows.Storage`** namespace. Thanks to this class, you can perform common operations such as getting all the files inside a folder, creating a subfolder, creating a file, and so on.

The **`StorageFolder`** class exposes a static method that is particularly useful with the Windows App SDK called **`GetFolderFromPathAsync()`**. This API represents one of the key differences between UWP applications and Windows App SDK applications: the first run inside a sandbox, so we can't freely access any file or folder inside the system; the latter, instead, run in full thrust and, as such, we can use this API to convert any path into a **`StorageFolder`** object, as in the following sample:

```
private async Task GetFolderAsync()
{
    var path = Environment.GetFolderPath(Environment.
      SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder.
      GetFolderFromPathAsync(path);
    Console.WriteLine(folder.Path);
}
```

In this example, we are combining the .NET APIs (the ones exposed by the **`Environment`** class) with the Universal Platform APIs to get a **`StorageFolder`** object that maps the local application data folder of the user.

We can use the **`StorageFolder`** object to work with every sub-folder of the current folder, either by opening a specific one (using the **`GetFolderAsync()`** method passing, as a parameter, the folder name) or by querying all the available sub-folders (using the **`GetFoldersAsync()`** method). The following example expands the previous snippet by retrieving a list of all the sub-folders of the local application data folder and printing their paths on the console:

```
private async Task GetFolderAsync()
{
    var path = Environment.GetFolderPath
      (Environment.SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder
      .GetFolderFromPathAsync(path);
    var folders = await folder.GetFoldersAsync();
    foreach (var folderItem in folders)
    {
        Console.WriteLine(folderItem.Path);
    }
}
```

However, once we have a **`StorageFolder`** object, the most interesting opportunity is to work with files that belong to the folder.

Working with files

The **`StorageFolder`** object, in a similar way we have seen for folders, exposes methods to get access to the stored files, either directly (by calling **`GetFileAsync()`** and passing the name) or by querying all the files (using the **`GetFilesAsync()`** method). Either way, files are represented by another class of the UWP called **`StorageFile`**. The following sample shows how to use the second option to list all the files that are stored inside the local application data folder and to print, on the console, their full paths:

```
private async Task GetFilesAsync()
{
    var path = Environment.GetFolderPath(Environment.
      SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder.
      GetFolderFromPathAsync(path);
    var files = await folder.GetFilesAsync();
    foreach (StorageFile file in files)
    {
```

```
            Console.WriteLine(file.Path);
        }
    }
```

Since we are working on a Win32 application, we can also directly convert a file path to a **`StorageFile`** object, as we have seen with the **`StorageFolder`** class:

```
    private async Task GetFilesFromPathAsync()
    {
        var path = Environment.GetFolderPath(Environment.
          SpecialFolder.LocalApplicationData);
        string filePath = @$"{path}\MyFile.txt";
        StorageFile file = await StorageFile.
          GetFileFromPathAsync(filePath);
    }
```

Another way to get a reference to a file is to create one by using the **`CreateFileAsync()`** method passing, as a parameter, the name of the file and, optionally, the behavior you want to adopt if the file already exists, as in the following example:

```
    private async Task CreateFileAsync()
    {
        var path = Environment.GetFolderPath
          (Environment.SpecialFolder.LocalApplicationData);
        var folder = await StorageFolder
          .GetFolderFromPathAsync(path);
        StorageFile file = await
    folder.CreateFileAsync("file.txt",
          CreationCollisionOption.ReplaceExisting);
    }
```

Once you have a **`StorageFile`** object, you have access to multiple asynchronous APIs to perform basic operations on files like the following:

- Using **`CopyAsync()`** and **`CopyAndReplaceAsync()`** to copy the file to another location. You must pass, as a parameter, a **`StorageFolder`**

object that maps the folder you want to use as the destination.

- Using **`MoveAsync()`** and **`MoveAndReplaceAsync()`** to move the file to another location. Also, in this case, you must pass as a parameter a **`StorageFolder`** object that maps the folder to use as the destination. Optionally, you can also pass a new name for the file.
- Using **`RenameAsync()`** to rename the file, passing as a parameter the new name.
- Using **`DeleteAsync()`** to delete the file.

One of the most common operations when you work with files is treating them as streams so that you can read their content or write data inside them. The UWP uses an interface called **`IRandomAccessStream()`** to manipulate streams, which you can obtain by calling the **`OpenAsync()`** method exposed by the **`StorageFile`** class. The following sample shows how you can use this type of stream to write some content into a text file:

```
private async Task WriteFileAsync()
{
    var path = Environment.GetFolderPath(Environment.
      SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder.
      GetFolderFromPathAsync(path);
    StorageFile file = await
folder.CreateFileAsync("file.txt",
      CreationCollisionOption.ReplaceExisting);
    IRandomAccessStream randomAccessStream = await
file.
      OpenAsync(FileAccessMode.ReadWrite);
    using (DataWriter writer = new
      DataWriter(randomAccessStream.GetOutputStreamAt(0
)))
    {
        writer.WriteString("Hello world!");
        await writer.StoreAsync();
```

```
    }
}
```

We open the file in write mode (by passing, as a parameter of the **`OpenAsync()`** method, the value **`ReadWrite`** of the **`FileAccess`** enumerator) and then we use the **`DataWriter`** class as a wrapper for the stream (starting from the beginning of the file). Thanks to this class, we can write some text inside. The example shows the usage of the **`WriteString()`** method, but the **`DataWriter`** class exposes many other methods for every other data type, such as **`WriteBytes()`**, **`WriteBoolean()`**, **`WriteTimeSpan()`**, and many more.

If instead, you prefer to work with the traditional **`Stream`** class exposed by .NET (which belongs to the **`System.IO`** namespace), you can use one of the two available additional methods, based on the type of operation you want to perform: **`OpenStreamForReadAsync()`** and **`OpenStreamForWriteASync()`**. The following snippet shows the same writing example, but implemented using the .NET streams:

```
private async Task WriteFileAsync()
{
        var path =
Environment.GetFolderPath(Environment.
          SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder.
      GetFolderFromPathAsync(path);
    StorageFile file = await
folder.CreateFileAsync("file.txt",
      CreationCollisionOption.ReplaceExisting);
    StorageFile file = await StorageFile.
      GetFileFromPathAsync(filePath);
    using (var stream = await
file.OpenStreamForWriteAsync())
    {
        StreamWriter writer = new StreamWriter(stream);
        writer.WriteLine("Hello world");
```

```
            await writer.FlushAsync();
        }
    }
```

In some cases, however, you won't need to work with streams to read and write data to a file, thanks to a series of helpers that are included directly in the UWP or in the Windows Community Toolkit. Let's start with the first scenario.

The **`Windows.Storage`** namespace includes a class called **`FileIO`**, which exposes a series of static methods to read and write the most common data types to a file, like the following:

- **`ReadTextAsync()`** and **`WriteTextAsync()`**, to read and write text content.
- **`WriteBuffer()`** and **`ReadBuffer()`** to read and write binary data.
- **`AppendTextAsync()`** to append text to an existing file.

All these methods work with a **`StorageFile`** object. The following sample shows how the previous writing sample can be simplified by using the **`FileIO`** class instead of streams:

```
private async Task WriteFileAsync()
{
    var path = Environment.GetFolderPath(Environment.
      SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder.
      GetFolderFromPathAsync(path);
    StorageFile file = await
folder.CreateFileAsync("file.txt",
      CreationCollisionOption.ReplaceExisting);
    //write the content to the file
    await FileIO.WriteTextAsync(file, "Hello world");
    //read the content from the file
    string content = await FileIO.ReadTextAsync(file);
}
```

Let's see now, instead, how the Windows Community Toolkit can help you to improve working with files and folders. As the first step, you must install the **`CommunityToolkit.WinUI`** package in your project from NuGet. Please note that if you have a WPF or Windows Forms application, this will introduce a dependency to the Windows App SDK, which wasn't required to perform all the other operations we have seen so far.

A scenario that makes the usage of the toolkit a great help is downloading a file from the network into your application. This scenario, typically, involves the usage of two chained operations: using the **`HttpClient`** class to download the stream of the file and then using the storage APIs to save it into a file. Thanks to the **`StreamHelper`** class, which belongs to the **`CommunityToolkit.WinUI.Helpers`** namespace, we can instead easily perform the task with a single line of code:

```
private async Task WriteFileAsync()
{
    var path = Environment.GetFolderPath(Environment.
      SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder.
      GetFolderFromPathAsync(path);
    StorageFile file = await
folder.CreateFileAsync("image.
      jpg", CreationCollisionOption.ReplaceExisting);
    await
StreamHelper.GetHttpStreamToStorageFileAsync(new
      Uri("https://www.mywebsite.com/image.jpg"),
file);
}
```

We use the **`GetHttpStreamToStorageFileAsync()`** method, passing as parameters the URL of the file and the **`StorageFile`** object where we want to save the downloaded content.

If you need to work with the content of the file directly in memory, instead, you can use the **`GettHttpStreamAsync()`** method, which will return an **`IRandomAccessStream`** object.

Another important scenario that isn't supported by the native APIs, but is provided by the toolkit, is checking whether the file already exists. Once you add the **`CommunityToolkit.WinUI.Helpers`** namespace to your file, the **`StorageFile`** class will expose a new extension method called **`FileExistsAsync()`**, which you can use as follows:

```
private async Task WriteFileAsync()
{
    var path = Environment.GetFolderPath
      (Environment.SpecialFolder.LocalApplicationData);
    var folder = await StorageFolder
      .GetFolderFromPathAsync(path);
    bool isExisting = await
folder.FileExistsAsync("image.
      jpg");
    if (!isExisting)
    {
        StorageFile file = await folder.CreateFileAsync
          ("image.jpg", CreationCollisionOption
            .ReplaceExisting);
        await
StreamHelper.GetHttpStreamToStorageFileAsync
          (new Uri("https://www.mywebsite.com/
            image.jpg"), file);
    }
}
```

In this example, we download an image from the web only if a file named **`image.jpg`** doesn't already exist in the local application data.

The Windows Community Toolkit also gives you some other interesting helpers, but before talking about them, we need to introduce a

new concept: local storage.

Using the local storage in packaged apps

When you package your application with MSIX, you get an identity, which gives you access to a broader set of features and integration with the Windows ecosystem. One of them is access to special storage, which is local and belongs only to the application itself. This storage is created in the `%LOCALAPPDATA%\Packages` folder. Each packaged application will have its own sub-folder with a name something like the **Package Family Name** of the application, which is the unique identifier assigned by Windows to packaged apps. This means that, if you try to use this special storage in an unpackaged application, you will get an exception.

The advantage of this storage is that it follows the same life cycle as your application, making it a great fit for storing all the files and folders that it wouldn't make sense to keep once the application is removed: configuration files, logs, local databases, and so on. When the application is uninstalled, Windows will also take care of removing the local storage.

This storage can be accessed using the `Windows.Storage.ApplicationData` class, which exposes a static instance called `Current`. Windows supports several types of local storage, which are mapped with different sub-folders. Currently, the two relevant types are the following:

- `Local`, which is exposed through the `ApplicationData.Current.LocalFolder` class. This is a general-purpose folder to store any kind of file or folder that belongs to your application and that must stay available until the application is uninstalled.
- `Temporary`, which is exposed through the `ApplicatonData.Current.TemporaryFolder` class. This folder is a

great fit for all the types of data that can be temporary, such as images that are cached to save bandwidth. When Windows is running low on disk space (or when the user performs a disk cleanup using the integrated Windows tool), it will clean up the temporary storage of all the applications. As you can imagine, this isn't the right place to put critical data.

Both these folders are mapped using the **`StorageFolder`** class, which means that you can use all the APIs we have seen so far to work with them. The following example shows the creation of a text file inside the local storage of the application:

```
private async Task WriteFileAsync()
{
    StorageFile file = await ApplicationData.Current.
      LocalFolder.CreateFileAsync("file.txt",
        CreationCollisionOption.ReplaceExisting);
    await FileIO.WriteTextAsync(file, "Hello world");
}
```

When you decide to use this special type of storage, the Windows Community Toolkit can help you with various methods exposed by the **`StorageFileHelper`** class. For example, the previous example can be simplified with a single line of code:

```
private async Task WriteFileAsync()
{
    //write a text file in the local storage
    await StorageFileHelper.WriteTextToLocalFileAsync
      ("Hello world", "file.txt",
CreationCollisionOption
.        ReplaceExisting);
    //write a text file in the temporary storage
    await
StorageFileHelper.WriteTextToLocalCacheFileAsync
```

```
        ("Hello world", "file.txt",
  CreationCollisionOption
          .ReplaceExisting);
  }
```

As you can see, the **`StorageFileHelper`** class exposes methods that you can use to directly write data to specific storage. In the previous example, we're using **`WriteTextToLocalFileAsync()`** to write text content directly into a file in the local storage, while **`WriteTextToLocalCacheFileAsync()`** does the same, but on a file in the temporary storage.

If instead, you need to create files that should survive the uninstallation of your application (for example, when you uninstall Office, Windows doesn't delete all your existing Word documents and Excel spreadsheets), you should use other system folders, such as the **`Documents`** library. Since your application is based on the Win32 ecosystem, the APIs we have seen so far don't have the same limitations as the sandbox used by UWP apps anymore. The Windows Community Toolkit can help you to simplify these scenarios as well, thanks to the other methods offered by the **`StorageFileHelper`** class. Consider the following sample:

```
  private async Task WriteFileAsync()
  {
      string path =
  Environment.GetFolderPath(Environment.
        SpecialFolder.MyDocuments);
      var folder = await StorageFolder.
        GetFolderFromPathAsync(path);
      await
  StorageFileHelper.WriteTextToFileAsync(folder, "Hello
        world", "File.txt");
  }
```

We are creating a text file inside the **`Documents`** folder of the user, retrieved using the **`Environment`** class offered by .NET. Thanks to the

StorageFileHelper class, we don't have to create the file first and then write the text, but we can do it with one line of code by using the **WriteTextToFileAsync()** method, passing as parameters, the **StorageFolder** object where we want to create the file, the text, and the filename. All these samples are based on text files but, of course, you can store binary data (such as an image) using the equivalent methods prefixed by **WriteBytes** (such as **WriteBytesToFileAsync()** or **WriteBytesToLocalFileAsync()**).

When your application is packaged, you also get the choice to use a special application URI to identify a file:

- If you want to reference a file inside the package, you can use the **ms-appx** protocol. For example, let's say that in your Visual Studio project, you have a configuration file called **config.json**, placed inside the root. You can access this file using the URI **ms-appx:///config.json**.
- If you want to reference a file inside the local storage, you can use the **ms-appdata** protocol. For example, if the same **config.json** file is stored inside the root of the local storage, you can reference it with the URI **ms-appdata:///local/config.json**.

The **StorageFile** class offers a static method to retrieve a reference to a file starting from the application URI called **GetFileFromApplicationUriAsync()**, which you can use as in the following sample:

```
StorageFile file = await
StorageFile.GetFileFromApplicationUriA
sync("ms-appx:///config.json");
```

In the next section, we're going to see another useful feature provided by local storage: the ability to store settings.

Using local storage to store settings

Another common requirement for applications when it comes to working with storage is to manage settings. It's very likely, in fact, that your application enables your users to customize one or more options. If your application is packaged, local storage gives you an excellent way to store settings, since it removes all the complexity of having to support a complex data structure, such as a database or a JSON file.

Local settings, in fact, can be easily managed through a dictionary, which stores key-value pairs. The following example shows how we can use this feature to store the theme selected by the user:

```
private void OnWriteSettings()
{
    ApplicationData.Current.LocalSettings.Values["Theme
"] =
      "Dark";
    var value = ApplicationData.Current.LocalSettings.
      Values["Theme"].ToString();
}
```

The collection of settings is exposed by the **`ApplicationData.Current.LocalSettings.Values`** property. In the first line of code, we create a new item with **`Theme`** as a key and **`Dark`** as a value. In the next line, we do the opposite operation: we retrieve the value of the item identified by the **`Theme`** key and we store it in a variable.

As you will notice, when you work with the local settings, you must manually manage the casting, since the **`Values`** collection can store any generic object. To simplify the code (and make it more readable), in this case, you can also use a helper provided by the Windows Community Toolkit called **`ApplicationDataStorageHelper`**. Let's see how we can simplify the previous code:

```
private void OnWriteSettings()
{
```

```
    var helper =
ApplicationDataStorageHelper.GetCurrent();
    //write the setting
    helper.Save<string>("Theme", "Dark");
    //read the setting
    var value = helper.Read<string>("Theme");
}
```

We get a reference to the **`ApplicationDataStorageHelper`** object by calling the **`GetCurrent()`** method. Then, we can save and read settings by using the **`Save()`** and **`Read()`** methods. Both methods support generics, so you don't have to manually cast the data type anymore. In the previous example, since the information about the theme is a string, we just use **`Read<string>()`** to directly get back a string object.

So far, we have seen the options we have to work with files in a programmatic way, without needing any user intervention. However, there are scenarios where we must ask for input from the user, such as the location where they want to save a file or to choose a file to import into the application. In the next section, we'll learn how to manage this scenario with pickers.

Working with file pickers

File pickers are important in applications since they provide a way for the user to load or save files in a specific location. When you use a picker, the application will open a dialog that enables users to explore their hard disk and look for a file to import into the app or a location to save a file created by the app. WPF and Windows Forms have their own classes to support file pickers. For example, in WPF you use the **`OpenFileDialog`** class. But what about WinUI? In this case, you can use the picker APIs that belong to the **`Windows.Storage.Pickers`** namespace. Let's see how to implement both open and save scenarios.

Using a picker to open a file

To import a file into your application, you can use the **`FileOpenPicker`** class. The usage compared to the UWP is a bit different since, in the case of a Win32 app, we must manually link the picker to our current window, through its native handle (HWND):

Here is an example of using the **`FileOpenPicker`** class in a WinUI application:

```
private async void OnPickFile(object sender,
RoutedEventArgs e)
{
    var filePicker = new FileOpenPicker();
    // Get the current window's HWND by passing in the
Window
      object
    var hwnd = WinRT.Interop.WindowNative.
      GetWindowHandle(this);
    // Associate e HWND with the file picker
    WinRT.Interop.InitializeWithWindow.Initialize(fileP
icker,
      hwnd);
    filePicker.FileTypeFilter.Add("*");
    StorageFile file = await
filePicker.PickSingleFileAsync();
    if (file != null)
{
      Console.WriteLine(file.Path);
}
```

First, you retrieve the HWND using the **`WinRT.Interop.WindowNative.GetWindowHandle()`** method. In this example, we're assuming the code is running directly inside a **`Window`**, so we can pass this as a parameter. If you are executing this code in another class (for example, a page or a helper), you must pass a reference to the **`Window`** object you have declared in the **`App`** class.

Other than that, the usage of the class is straightforward:

1. First, you use the **`FileTypeFilter`** collection to add all the extensions you want to support. The dialog will display only files that respect these criteria. In the example, we're passing a wildcard (**`*`**), which means that we're going to display all the files.
2. Then, you call the **`PickSingleFileAsync()`** method, which will return a **`StorageFile`** object. If you need to open multiple files at once, you can use the **`PickMultipleFilesAsync()`** method, which, in this case, will return a collection of **`StorageFile`** objects, one for each selected file.

Once you have a reference to the **`StorageFile`** object (or the objects, if you picked multiple files), you can use the APIs we have seen in the other sections of this chapter to perform further operations, such as reading its content. In the example, we're just printing on the console output the full file path.

The previous example is based on a WinUI application, but the same API can also be used in other platforms. The only difference is the way you retrieve the HWND, since **`WinRT.Interop.WindowNative.GetWindowHandle()`** is a specific WinUI API. In WPF, for example, you would use the following code:

```
private async void OnPickFile(object sender,
RoutedEventArgs e)
{
    var filePicker = new FileOpenPicker();
    var hwnd = new
WindowInteropHelper(this).EnsureHandle();
    WinRT.Interop.InitializeWithWindow.Initialize(fileP
icker,
      hwnd);
    filePicker.FileTypeFilter.Add("*");
```

```
    var file = await
filePicker.PickSingleFileAsync();
    if (file != null)
{
      Console.WriteLine(file.Path);
}
```

As you will notice, everything is the same except for the highlighted line: in WPF, you use the **`EnsureHandle()`** method of the **`WindowInteropHelper`** class to retrieve the HWND of the window passed as a parameter.

The UWP also has an equivalent API for folders called **`FolderPicker`**, which exposes the **`PickSingleFolderAsync()`** method. The API works in the same way, with only the following differences:

- You don't need to specify the **`FileTypeFilter`** collection since folders don't have a type.
- The object type you get in return is **`StorageFolder`** instead of **`StorageFile`**.

Here is an example:

```
private async void OnPickFolder(object sender,
RoutedEventArgs
  e)
{
    var folderPicker = new FolderPicker();
    var hwnd = WinRT.Interop.WindowNative.
      GetWindowHandle(this);
    WinRT.Interop.InitializeWithWindow.Initialize(folde
rPicker,
      hwnd);
    StorageFolder folder = await folderPicker.
      PickSingleFolderAsync();
```

```
    if (file != null)
{
      Console.WriteLine(file.Path);
}
```

Once you have a **`StorageFolder`** object, you can perform all the operations we have seen in this chapter, such as listing the sub-folders or the included files or creating a new file.

Let's see now how we can use a picker to save a file, instead.

Using a picker to save a file

With a picker, you can enable users to choose where to save a file with content generated by your application. The class to use is called **`FileSavePicker`** and its usage is similar to the **`FileOpenPicker`** one we have just learned how to use. The only difference is that this time, the picker will take care of creating a new file and it will give us back a reference to it with a **`StorageFile`** object, which we can use to save the content.

Let's take a look at the following example:

```
private async void OnSaveFle(object sender,
RoutedEventArgs e)
{
    var filePicker = new FileSavePicker();
    var hwnd = WinRT.Interop.WindowNative.
      GetWindowHandle(this);

    WinRT.Interop.InitializeWithWindow.Initialize(fileP
icker,
      hwnd);
    filePicker.SuggestedFileName = "file.txt";
    filePicker.FileTypeChoices.Add("Text", new
List<string>() {
```

```
        ".txt" });
    StorageFile file = await
filePicker.PickSaveFileAsync();
    await FileIO.WriteTextAsync(file, "Hello world");
}
```

As for **`FileOpenPicker`**, in this case, we also have to first retrieve the native handle of the window and use it to initialize a new **`FileSavePicker`** object. The **`FileSavePicker`** class offers a few properties that we can use to customize the experience, such as **`SuggestedFileName`** to set the default name of the file. However, the critical one (without setting it, the picker will return an exception) is **`FileTypeChoices`**, which is a collection of the file types that your application can generate. In this example, we're assuming that the application can generate text files. In the end, we call **`PickSaveFileAsync()`**, which will trigger the Windows dialog. Once we have chosen the location to save the file and a name, we will get back a **`StorageFile`** object that we can fill with the content. In this example, we use the **`FileIO`** APIs to store sample text.

With this, we have completed our journey into exploring the storage APIs offered by the UWP, which can be a great companion for the .NET APIs, especially in WinUI applications. Now we can explore another option offered by Windows integration: sharing data across multiple applications.

Supporting the sharing contract

Sharing is one of the most common actions you perform on your computer. You don't always realize it because you do it multiple times per day, but when you copy some content and you paste it into another application, you're effectively sharing data across two different applications. However, there might be scenarios where a copy and paste might be fully effective: for example, you're copying a URL or some

other complex data and you would like the other application to understand it and treat it in the proper way.

To make this scenario more effective, Windows 10 has introduced a sharing contract, which acts as a standard way to share specific types of data (such as images, texts, and links) across different applications. Being a contract, it means that the two applications don't need to know each other:

- The source application will use the sharing APIs to share data, using one of the specific formats supported by the contract.
- The target application will register itself in the system as eligible to receive one or more types of data from the sharing contract.

Let's build a suite of applications to implement this scenario: a source one and a target one. We're going to use a WPF application based on .NET 6.0, to show you that you don't need to move to WinUI as a UI layer to leverage this feature. However, the same exact code will work also in a Windows Forms or WinUI application.

Let's start!

Building the source app

The source app is the simplest one to build since it doesn't need any special dependency. The only requirement is that, like for the other samples we have seen in this chapter, we must set **`TargetFramework`** of our WPF application a specific Windows 10 or 11 one, since the sharing contract is part of the UWP. As such, make sure that the **`TargetFramework`** property of your WPF (or Windows Forms project) looks like this:

```
<TargetFramework>net6.0-
windows10.0.19041</TargetFramework>
```

The next step is to create a helper class, which will enable us to use the sharing APIs from a Win32 application. If you already have experience building UWP applications, you will realize this is a different experience. This is what the helper class looks like:

```
public static class DataTransferManagerHelper
{
    static readonly Guid _dtm_iid = new
Guid(0xa5caee9b,
      0x8708, 0x49d1, 0x8d, 0x36, 0x67, 0xd2, 0x5a,
0x8d,
        0xa0, 0x0c);
    static IDataTransferManagerInterop
      DataTransferManagerInterop => DataTransferManager
        .As<IDataTransferManagerInterop>();
    public static DataTransferManager
GetForWindow(IntPtr hwnd)
    {
        IntPtr result;
        result =
DataTransferManagerInterop.GetForWindow(hwnd,
          _dtm_iid);
        DataTransferManager dataTransferManager =
          MarshalInterface<DataTransferManager>
            .FromAbi(result);
        return (dataTransferManager);
    }
    public static void ShowShareUIForWindow(IntPtr
hwnd)
    {
        DataTransferManagerInterop.ShowShareUIForWindow
(hwnd);
    }
    [ComImport]
    [Guid("3A3DCD6C-3EAB-43DC-BCDE-45671CE800C8")]
```

```
    [InterfaceType(ComInterfaceType.InterfaceIsIUnknown
)]
    public interface IDataTransferManagerInterop
    {
        IntPtr GetForWindow([In] IntPtr appWindow, [In]
ref
          Guid riid);
        void ShowShareUIForWindow(IntPtr appWindow);
    }
}
```

In a UWP application, this helper isn't required because the platform already provides the required hooks to access these APIs via Windows Runtime. We won't look at the class implementation in detail, since it's all boilerplate code required to import the proper COM interfaces and objects required by the sharing APIs.

What's interesting to see is how to use this API in our source application:

```
private void OnShareData(object sender, RoutedEventArgs
e)
{
    var myHwnd = new
WindowInteropHelper(this).EnsureHandle();
    var dataTransferManager =
DataTransferManagerHelper.
      GetForWindow(myHwnd);
    dataTransferManager.DataRequested += (obj, args) =>
    {
        args.Request.Data.SetText("This is a shared
text");
        args.Request.Data.Properties.Title = "Share
Example";
        args.Request.Data.Properties.Description = "A
          demonstration on how to share";
```

```
    };
    DataTransferManagerHelper.ShowShareUIForWindow(myHw
nd);
}
```

The way we trigger the preceding code is based on how and which kind of data we want to share from our application. In the previous example, we're assuming our application has a share button, which invokes the preceding event handler.

The first step is to retrieve the HWND, which is the native handler of the window. As we have learned in the section dedicated to file pickers, this is the only code that must be different based on the UI platform you're using: the preceding example, based on the **`WindowInteropHelper`** class, works with WPF and Windows Forms. If you're using WinUI, instead, you will need to use the following code to retrieve the HWND:

```
var myHwnd =
WinRT.Interop.WindowNative.GetWindowHandle(this);
```

The rest of the code is the same regardless of the UI platform. First, we obtain a **`DataTransferManager`** object for the current window, using the **`DataTransferManagerHelper`** class we previously created. Then we subscribe to the **`DataRequested`** event, which is triggered when the sharing operation is invoked. Inside the event handler, we must supply to the request the data we want to share, through the **`Request.Data`** property exposed by the event arguments.

In the previous example, we're sharing some text using the **`SetText()`** method. Other supported options are **`SetUri()`** to share a link or **`SetStorageItems()`** to share a file (passing, as a parameter, a collection of **`StorageFile`** objects, which we have learned how to use in this chapter). We also customize the properties of the sharing operation, by setting its **`Title`** and **`Description`**.

In the end, we trigger the sharing UI by calling the `ShowShareUIForWindow()` method exposed by the `DataTransferManagerHelper` class passing, once again, the HWND of the current window, which we have previously retrieved.

Even if we haven't developed the target application, we can already see the sharing contract in action, since in Windows there are many pre-installed apps that can act as a target, such as **Mail**:

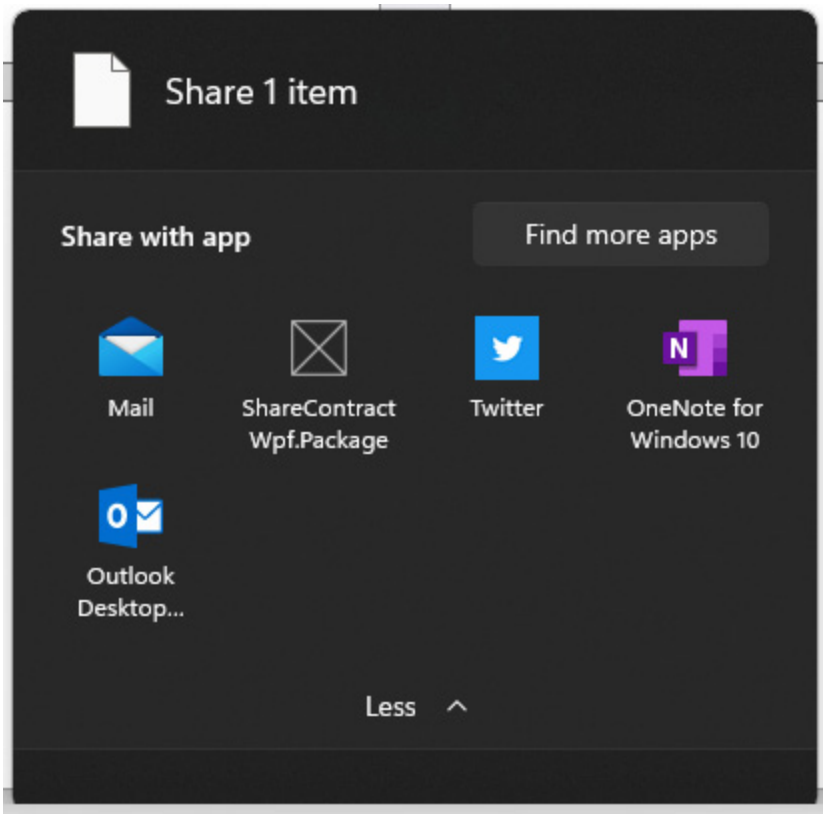

Figure 8.5 – The Share UI provided by Windows 11

Thanks to the Windows integration, you can also see a glimpse of how powerful this feature is. If you choose **Mail** as the target, for example, you can see how the information you have shared is automatically used to compose a new message: the title is set as a subject of the mail, while the text becomes the body of the mail.

Let's see now how we can build a similar experience.

Building the target app

Building the target app requires a bit more work since it's the one that will effectively share the content. If we take as an example the previous test we did, we realize that it's the Mail app that is in charge of the most complex work, since it needs to take the data shared by the source app, turn it into an email, and then provide a way for the user to send it.

NOTE

At the time of writing, acting as a share target is supported only by packaged apps. As such, if you're using WPF or Windows Forms, make sure to add the Windows Application Packaging Project to your solution; if you're using WinUI, instead, make sure to use the packaged model. If you're integrating the sharing contract into a WPF or Windows Forms application, you must also install the Windows App SDK NuGet package, as described in ***Chapter 1****, Getting Started with the Windows App SDK and WinUI. We'll need to use the Activation APIs, which are included in the runtime, to manage the application activation from a sharing operation.*

To recreate a similar experience in our application, as such, we need to have a dedicated window or page for the sharing operation. When the application is opened normally (by clicking on the icon in the **Start** menu, for example), the main page of the application will be displayed. When the application is opened through a sharing contract, instead, we'll redirect the user to a specific page of the application, which gives the user the option to share the content. The page could display a preview of the shared content; provide extra fields to add additional information; or have a button to complete the share operation.

For example, in our WPF application, we have created a window called **`ShareWindow`** dedicated to the sharing operation, which contains two **`TextBlock`** controls to display a preview of the sharing data and a button to perform the sharing:

```
<Window>
    <StackPanel>
        <TextBlock x:Name="txtTitle" />
        <TextBlock x:Name="txtSharedText" />
        <Button Content="Complete" Click="OnComplete"/>
    </StackPanel>
</Window>
```

The next step is to tweak our application's initialization code so that we can redirect the user to one page or another based on the scenario. In our example, since it's a WPF application, we can override the **`OnStartup()`** event of the **`App`** class. If it was a WinUI app, we would have leveraged the **`OnLaunched()`** event of the **`App`** class. Regardless of which is the entry point, however, the code we're going to execute is the same:

```
public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs
e)
    {
        Window Window;
        var instance = AppInstance.GetCurrent().
          GetActivatedEventArgs();
        if (instance.Kind == ExtendedActivationKind.
          ShareTarget)
        {
            window = new ShareWindow();
        }
        else
        {
            window = new MainWindow();
        }
        window.Show();
    }
}
```

Thanks to the **`AppInstance`** class, which comes from the Windows App SDK and belongs to the **`Microsoft.Windows.AppLifecycle`** namespace (we learned about it in ***Chapter 4***, *The Windows App SDK for a UWP Developer*), we can identify the activation event that led our application to be opened by calling the **`GetActivatedEventArgs()`** method. In the response, we can use the **`Kind`** property to distinguish between different activation events through the **`ExtendedActivationKind`** enumerator. If the value is **`ShareTarget`**, we are in a sharing scenario: we redirect the user to new **`ShareWindow`** we have just created for this purpose; otherwise, we redirect the user to the standard **`MainWindow`** instance, which is the default one.

In a WinUI application, rather than opening different **`Window`** (which could lead to a few challenges, since WinUI doesn't support multiple windows at the time of writing), we could have leveraged same **`MainWindow`** for both scenarios and then, using the navigation techniques we learned in ***Chapter 5***, *Designing Your Application*, we could have triggered navigation to a different page.

Now that the user is redirected to **`ShareWindow`**, let's see its implementation in code-behind:

```
public partial class ShareWindow : Window
{
    private ShareOperation shareOperation;
    public ShareWindow()
    {
        InitializeComponent();
    }
    private async void Window_Loaded(object sender,
      RoutedEventArgs e)
    {
        var instance = AppInstance.GetCurrent()
          .GetActivatedEventArgs();
        if (instance.Kind ==
```

```
            ExtendedActivationKind.ShareTarget)
        {
            var args = instance.Data as
              IShareTargetActivatedEventArgs;
            shareOperation = args.ShareOperation;
            string text = await args.ShareOperation
              .Data.GetTextAsync();
            string title = args.ShareOperation
              .Data.Properties.Title;
            txtSharedText.Text = text;
            txtTitle.Text = title;
        }
    }
    private void OnComplete(object sender,
RoutedEventArgs e)
    {
       // share the data
        shareOperation.ReportCompleted();
        Application.Current.Shutdown();
    }
}
```

We use the **`Loaded`** event of the Window class to retrieve all the information about the sharing operation. We again use the **`AppInstance`** class to retrieve the activation arguments and check whether it's indeed a **`ShareTarget`** scenario. This time, however, we move on to the next step, which is casting the **`Data`** property of the arguments to an **`IShareTargetActivatedEventArgs`** object, which is specific for sharing scenarios. Through this object, we can access the **`ShareOperation`** property, which contains all the information coming from the source application. In the previous example, we used the following:

- The **`GetTextAsync()`** method is exposed by the **`Data`** property to retrieve the shared text. Of course, the **`Data`** property exposes multi-

ple **`Get()`** methods for the different data types you can share, such as **`GetUrl()`** for links.

- The **`Properties`** object is exposed by the **`Data`** property to retrieve the title.

In the previous sample, we simply display this data in **`ShareWindow`** as a preview.

For the moment, however, we are just showing the user a glimpse of the data they have received from the source application. Now we need to build the logic to complete the sharing operation, connected to the **`Button`** control we have added in XAML. You won't find this logic in the previous sample, because it totally depends on the kind of application you have built: if it's a Twitter client, the button will post the tweet on your timeline; if it's a photo application, the image will be added to your gallery; and so on. However, regardless of the logic, it's important that you call the **`ReportCompleted()`** method exposed by the **`ShareOperation`** object at the end of the task so that Windows can complete the sharing operation.

If you have experience with the UWP, you will notice a different behavior in your Win32 application: it won't be automatically terminated once the sharing operation is completed. As such, it's up to you to implement the proper behavior based on your expectations. In the previous sample, we are terminating the WPF application by calling **`Application.Current.Shutdown()`**. In another scenario, you might want to keep the application alive and redirect the user to the main page.

We aren't done yet. We have implemented all the code we need, but we haven't told Windows that our application can act as a share target. We do this through the manifest of the application, by registering a specific extension. This is why to become a share target, your application must be packaged with MSIX: the extension is supported only by the manifest, unlike other activation paths (such as file and proto-

col), which are also supported by unpackaged apps, as we learned in ***Chapter 4***, *The Windows App SDK for a UWP Developer*.

Double-click on the **`Package.appxmanifest`** file in your Windows Application Packaging Project and move to the **Declarations** section. From the dropdown, add the **Share Target** declaration and, in the **Data formats** section, add a new entry for each data type you support as part of the sharing. In our example, where we support text, we add **Text** as a data format as shown in the following screenshot:

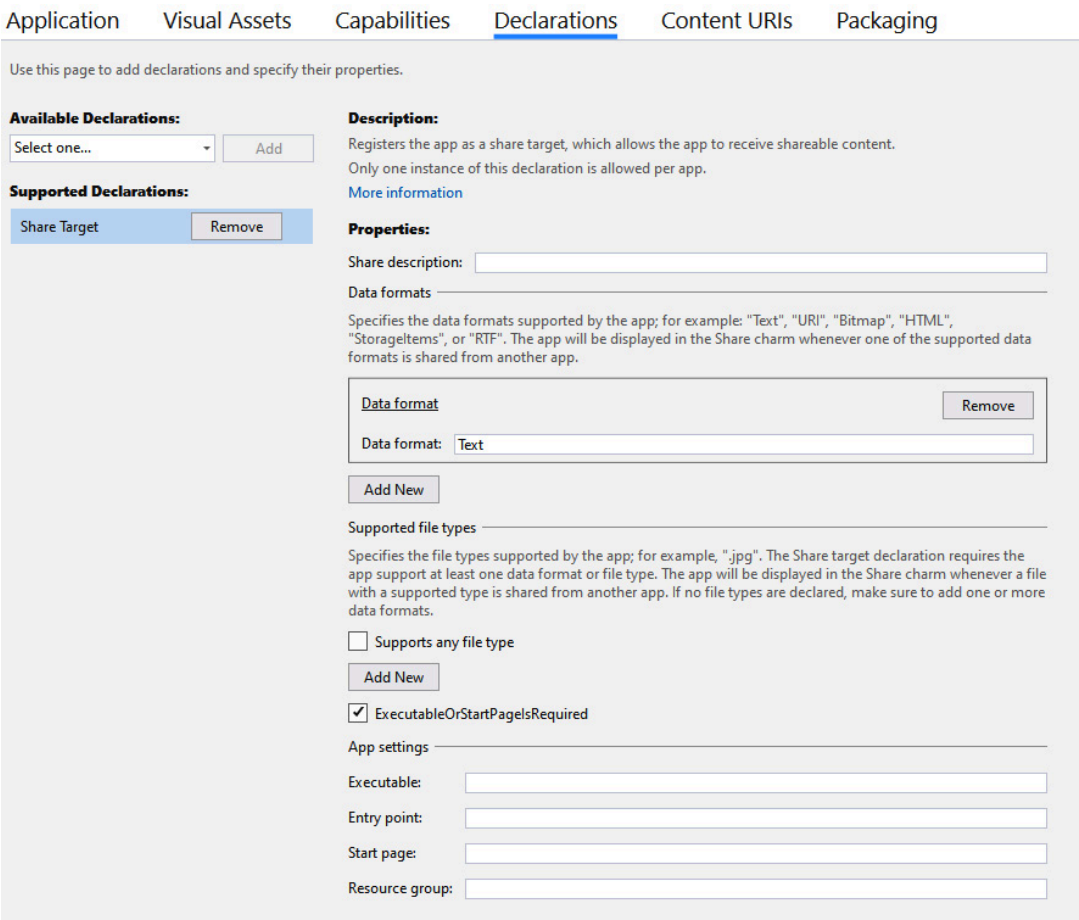

Figure 8.6 – Registering an application in the manifest to act as a share target

Now, after having deployed the share target application, you can test the outcome of your work by again launching the shared source one. If you have properly implemented all the components, you will see your application is listed as a potential target in the share UI.

We have completed our journey into the sharing contract. Thanks to this section, we have learned how we can share data across multiple applications roughly: thanks to the contract, we just need to define the type of data we want to share or that we can manage, and Windows will take care of connecting the dots by opening a communication channel.

Let's now move on to the final topic of this chapter: creating hybrid experiences between the web and the desktop ecosystem.

Integrating web experiences in your desktop application

The importance of the web has significantly increased over the last decade, and it keeps growing day by day. Daily, we perform multiple tasks using web applications: from making a money transfer from our bank account to sharing a document with a co-worker using a collaborative platform.

As such, developers often need to create hybrid applications that can deliver the best of both worlds: reusing the investments they made in web applications but enhancing them with the powerful capabilities offered by a native platform. Today, in your everyday job, you use many of these apps. A good example of this approach is Microsoft Teams: being a native app, it can take advantage of the native capabilities of Windows, such as push notifications and audio and video integration. At the same time, it can integrate a wide range of web experiences, from editing Office documents to getting real-time information from GitHub; from hosting Power Apps to displaying complex Power BI dashboards.

.NET platforms such as WPF and Windows Forms have provided the option to integrate web experiences for a long time thanks to the WebBrowser control, which you can use to host web applications in-

side your desktop application. However, there's a catch that still holds true even with the most recent revisions of the .NET ecosystem, such as .NET 6: the WebBrowser control uses the Internet Explorer engine to render web content. This means that the control isn't a good fit to host modern web experiences, since Internet Explorer uses an outdated engine that doesn't support all the latest innovations in web platforms, including the latest HTML, JavaScript, and CSS features.

For this reason, Microsoft has created a new control called WebView2, based on the same Chromium engine used by the new Microsoft Edge, which supports all the latest features of the web ecosystem. This control is available for all the Microsoft development platforms: in the case of WinUI, it's already integrated, while for the others, you must install a dedicated package.

Let's learn more in the next section!

Adding the WebView2 control to your page

The starting point of building a hybrid application with the **`WebView2`** control is different based on the development platform you have chosen:

- If you are building a WinUI application with the Windows App SDK, **`WebView2`** is already integrated. You won't have to do anything special to use it; you can just add it to your page as you do with any other standard control like **`Button`** or **`TextBlock`**, as in the following example:

```
<Window>
    <Grid>
        <WebView2 Source="http://www.packtpub.com"
          x:Name="MyWebView" />
    </Grid>
</Window>
```

- If you are building a WPF or Windows Forms application, you must first install a dedicated NuGet package. Please note that **`WebView2`** is supported both by .NET 5/6 applications and full .NET Framework ones. Right-click on your project, choose **Manage NuGet packages**, and install the package with **Microsoft.Web.WebView2** as the identifier. The main difference with WinUI is that, in this case, since the control comes from an external library, you will have to add an explicit reference. In the case of Windows Forms, you will find it in the designer toolbox; in the case of WPF, you must declare the **`Microsoft.Web.WebView2.Wpf`** namespace in your **`Window`** and then use it as a prefix, as in the following example:

```
<Window
        xmlns:wv2="clr-
namespace:Microsoft.Web.WebView2.
          Wpf;assembly=Microsoft.Web.WebView2.Wpf"
    <Grid>
        <wv2:WebView2
Source="http://www.packtpub.com"
          x:Name="MyWebView" />
    </Grid>
</Window>
```

Once you have added the control to your page, you can start working with it. The APIs exposed by the control are the same regardless of the UI platform of your choice.

In the previous snippet of code, you have already seen a glimpse of the basic usage of the control. By setting the **`Source`** property to a URL, the corresponding web application will be rendered inside the control. The following shows an example of such usage:

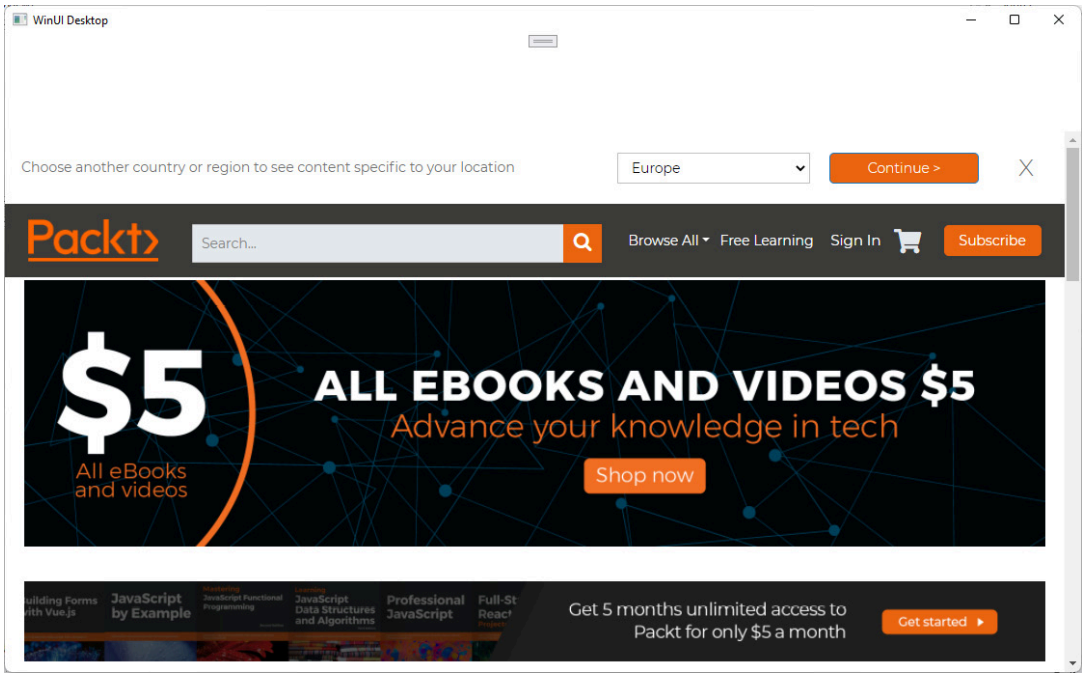

Figure 8.7 – A desktop WinUI application that is hosting the Packt website using the WebView2 control

The same goal can be achieved in code if you want to recreate a browser-like experience in your application:

```
private async void OnNavigateUrl(object sender,
  RoutedEventArgs e)
{
    await MyWebView.EnsureCoreWebView2Async();
    MyWebView.CoreWebView2.Navigate("http://www.packtpu
b.com");
}
```

The **`WebView2`** control (**`MyWebView`** is the name assigned to the control in XAML) exposes a property called **`CoreWebView2`**, which gives you access to all the advanced APIs exposed by the control. Before using it, it's always important to call the asynchronous **`EnsureCoreWebView2Async()`** method, which makes sure that the property has been correctly initialized. The **`Navigate()`** method is the one you can use to trigger navigation to a specific URL. You can also use the **`WebView2`** control to load local HTML content, by passing the HTML string to the **`NavigateToString()`** method:

```
private async void OnNavigateString(object sender,
  RoutedEventArgs e)
{
    await MyWebView.EnsureCoreWebView2Async();
    WebView.CoreWebView2.NavigateToString("<p>Hello
world!</
      p>");
}
```

Lastly, you can manage the life cycle of your web applications directly from the control by subscribing to one of the many available events, such as the following:

- **`NavigationStarting`**, when you trigger navigation to another URL (or when the page itself triggers a redirect).
- **`SourceChanged`**, when the **`Source`** property is set to another value.
- **`ContentLoading`**, when the content of the web page begins to load.
- **`NavigationCompleted`**, when the navigation process has been completed.

Many of these events give you access to advanced information about the HTTP request. For example, you can use the **`NavigationStarting`** event to inspect the destination URL, the request headers and, eventually, cancel the operation. Look at the following example:

```
<WebView2 x:Name="MyWebView"
NavigationStarting="WebView_
  NavigationStarting" />
```

In XAML, we have subscribed to the **`NavigationStarting`** event, which is managed by the following event handler:

```
private void MyWebView_NavigationStarting(WebView2
  sender, Microsoft.Web.WebView2.Core.
    CoreWebView2NavigationStartingEventArgs args)
{
    if (args.Uri == "https://www.microsoft.com")
```

```
        {
            args.Cancel = true;
            Console.WriteLine("This website is blocked!");
        }
    }
```

Through the event arguments, we can inspect the **`Uri`** property to see where the user is being redirected. In this example, we're simulating a policy that blocks non-allowed websites. If the URI matches with the Microsoft website, we set the **`Cancel`** property to **`true`**, which will abort the navigation.

These are some of the basic concepts that you can implement with the **`WebView2`** control. However, to really support hybrid scenarios, we need to move to the next level and learn how we can deeply integrate our web applications with the native experience.

Enabling interactions between the native and web layers

To fully support the creation of true hybrid apps, the **`WebView2`** control supports two powerful scenarios:

- Enabling web applications to send information to the native layer
- Enabling native applications to send information to the web layer

Let's see how we can implement both scenarios.

Communicating from the web app to the native app

The **`WebView2`** control can receive messages from the web application through a JavaScript API exposed by the browser called **`window.chrome.webview.postMessage()`**. Let's use this API to build a simple application that, given a specific city, returns its coordinates. The city's name will be sent by the web application, while the geocod-

ing operation will be performed using the location APIs we have learned how to use in this chapter.

Let's start with the web example:

```
<a
onclick="window.chrome.webview.postMessage('Milan')">Ge
t
  coordinates</a>
```

This is a snippet of HTML code that renders a hyperlink that, when it's clicked, invokes the **`postMessage()`** function passing, as a parameter, the city name (in this example, it's hardcoded; in a real app, it would be coming from a text field in a web page). When your web application is running in a traditional browser, this API won't be effective: you will get an error because the **`postMessage()`** function is **`undefined`**. As such, if your application is expected to also run in a browser, you should check if this function really exists and supply an alternative path.

When the web application is running inside a **`WebView2`** control, instead, it will trigger an event that you can intercept in the native layer, as the following example shows:

```
public sealed partial class MainWindow : Window
{
    public MainWindow()
    {
        this.InitializeComponent();
        MyWebView.EnsureCoreWebView2Async();
        MyWebView.CoreWebView2.WebMessageReceived +=
          CoreWebView2_WebMessageReceived;
    }
    private async void CoreWebView2_WebMessageReceived
      (CoreWebView2 sender,
        CoreWebView2
```

```
            WebMessageReceivedEventArgs args)
      {
    string city = args.TryGetWebMessageAsString();
    var result = await
  MapLocationFinder.FindLocationsAsync(city,
      null);
    Console.WriteLine($"Latitude: {result.Locations[0]
      .Point.Position.Latitude} - Longitude: {result
        .Locations[0].Point.Position.Longitude}");
       }
  }
```

The event is called **`WebMessageReceived`** and it's exposed by the **`CoreWebView2`** property of the **`WebView2`** control. Inside the event handler, we can use the **`TryGetWebMessageAsString()`** method exposed by the event arguments to retrieve the message coming from the JavaScript function. In our example, the application will receive the name of a city (Milan), which we have passed to the **`postMessage()`** function.

Since the **`WebMessageReceived`** event is handled in the native layer, we have access to the entire Windows API ecosystem. In the example, we're using the **`MapLocationFinder`** class (included in the **`Windows.Services.Maps`** namespace) to perform a geocoding: we call the **`FindLocationAsync()`** method passing the name of the city and we get back a collection of results (inside the **`Locations`** property). We grab the first item and we output to the console its latitude and longitude.

This was just an example, but the possibilities are unlimited since we are running this code in the client application. An action executed on the web page can execute any native Windows operation, from displaying a notification to connecting to a Bluetooth device; from saving a file on the disk to triggering an operation in the background.

Let's now move on to the opposite scenario.

Communicating from the native app to the web app

There are two approaches you can use to let your .NET application send information to the web application. The first one is by using messages, in a comparable way to what we did in the previous section (but in the opposite direction).

In your C# code, you can send a message using the **`PostWebMessageAsString()`** method (if you need to send a string) or the **`PostWebMessageAsJson()`** one (if you need to send a more complex structure described with JSON). The following example sends a string to the web application:

```
private async void OnButtonClicked(object sender,
  RoutedEventArgs e)
{
    Geolocator locator = new Geolocator();
    var result = await locator.GetGeopositionAsync();
    string message = $"Latitude:
{result.Coordinate.Point
      .Position.Latitude} - Longitude:
{result.Coordinate
        .Point.Position.Longitude}";
    await WebView.EnsureCoreWebView2Async();
    webView.CoreWebView2.PostWebMessageAsString(message
);
}
```

The first part of the code is using the **`Geolocator`** class we learned how to use in this chapter to retrieve the position of the user and to compose a message with the latitude and the longitude returned by location APIs. Then, by calling the **`PostWebMessageAsString()`** method exposed by the **`CoreWebView2`** object, we send this information to the web application.

On the web side, we can listen to incoming messages from the native layer with an event listener, as in the following example:

```
<script>
    window.chrome.webview.addEventListener('message',
arg => {
        window.alert(arg.data);
    });
</script>
```

We subscribe to the event called **`message`** exposed by the **`window.chrome.webview`** object. Inside the event handler, we can retrieve from the **`data`** property of the arguments the message sent by the native layer. In this example, we're just displaying it to the user using a web popup, by passing it to the **`window.alert()`** function supported by every browser.

Through the CoreWebView2 object, you can also directly invoke JavaScript APIs that are exposed by the page or the browser. The following sample will produce the same outcome as the previous one, but implemented differently:

```
private async void OnButtonClicked(object sender,
  RoutedEventArgs e)
{
    Geolocator locator = new Geolocator();
    var result = await locator.GetGeopositionAsync();
      string message = $"Latitude:
{result.Coordinate.Point
        .Position.Latitude} - Longitude:
        {result.Coordinate.Point.Position.Longitude}";
    await WebView.EnsureCoreWebView2Async();
    await
WebView.CoreWebView2.ExecuteScriptAsync($"window.
      alert('{message}')");
}
```

In this case, we don't need to add any code to our web application. We use the **`ExecuteScriptAsync()`** method exposed by the **`CoreWebView2`** object to directly execute the **`window.alert()`** function exposed by the browser:

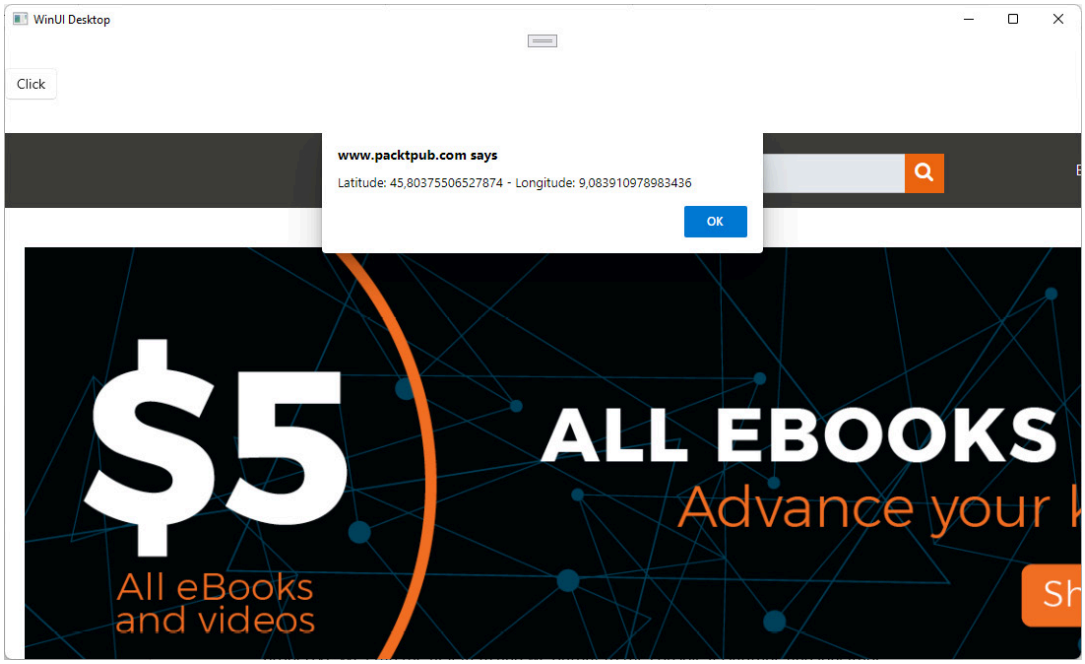

Figure 8.8 – The JavaScript function to display a web popup has been triggered from the native WinUI application

When you compile your application in debug mode, the **`WebView2`** control will give you access to the same developer tools that are available in Microsoft Edge. When your application is running, make sure the focus is on the web application and press *F12*: the developer tools will open up in another window, giving you the option to debug the JavaScript code, inspect the HTML, capture network traces, and much more:

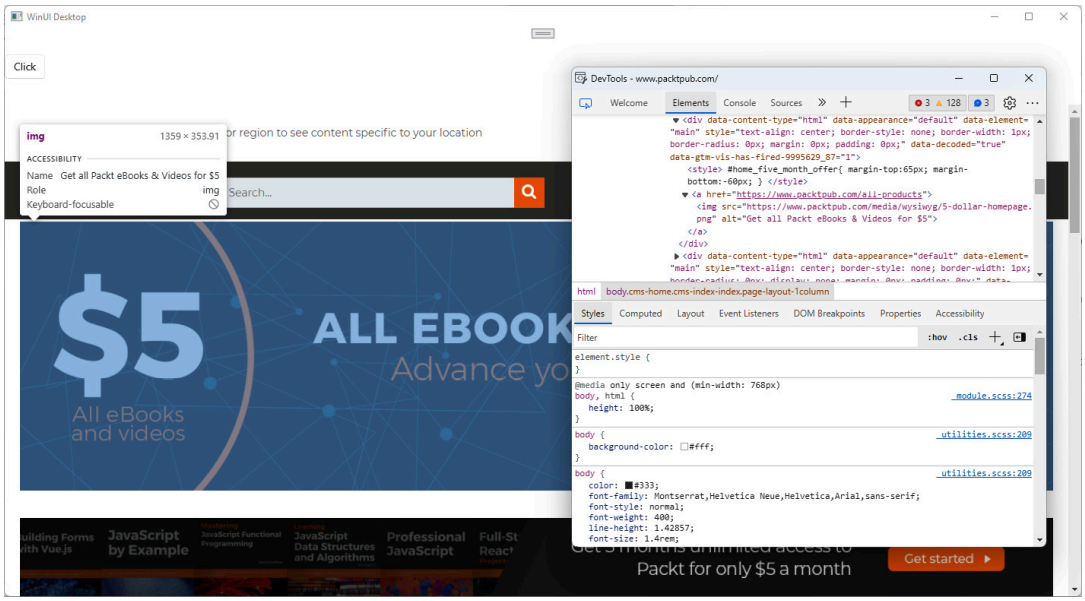

Figure 8.9 – The web developer tools enabled for a WebView2 control

Before completing our journey into building hybrid apps, we must introduce an important topic: the **`WebView2`** runtime.

Distributing the WebView2 runtime with your applications

Despite the **`WebView2`** control using the same engine as Edge, it isn't the browser doing the rendering, but a dedicated runtime that is optimized for hybrid scenarios called **`WebView2`** Runtime.

If your customers are running Windows 11, you're good to go: the **`WebView2`** runtime comes preinstalled, removing from you any responsibility for installing it and keeping it up to date. If your customers are running Windows 10, instead, users might not have the **`WebView2`** runtime already installed on their machine. As such, it's important to make sure that the runtime is installed before your application starts.

The WebView2 runtime supports two deployment techniques:

- **Evergreen version**: This is the suggested approach for most developers. The runtime is installed system-wide, and it's automatically kept updated by Windows.

- **Fixed version**: In this scenario, a specific version of the runtime is hard-linked to your application, and it's distributed as part of your binaries. This approach is a good fit for a critical application that must stay unchanged most of the time since you are in control of updates. The `WebView2` runtime will never be updated unless you choose to do so by releasing an updated version of the application that includes a newer version. As such, there are no chances that a `WebView2` update might potentially break or change the behaviors you have implemented.

Let's see some more details about the two distribution techniques.

Distributing the evergreen version runtime

The evergreen runtime comes as a standalone installer, which you can integrate into your deployment workflow. If you remember what we learned in ***Chapter 1***, *Getting Started with the Windows App SDK and WinUI*, it's the same approach that we must adopt when we want to distribute unpackaged apps that use Windows App SDK. Since it's a traditional setup, which can run unattended by using the `/silent /install` switches, you can easily integrate it into your MSI installer or your PowerShell deployment script.

There are three options to perform the integration:

- You can use a bootstrapper, which is a very small setup that will download the most recent version directly from the internet as part of the installation. The bootstrapper itself can be downloaded as part of the installation process. All you have to do is to go to the official `WebView2` runtime download page at **https://developer.microsoft.com/en-us/microsoft-edge/webview2/#download-section** and, under the **Evergreen Bootstrapper** section, click on **Get The Link** button. You will get a URL, which you can integrate into your installer to download the bootstrapper and execute it, before installing the main application.

- You can use a bootstrapper but include it in your installer instead of downloading it in real time. In this case, from the same **`WebView2`** runtime download page, you can click the **Download** button to download the **`MicrosoftEdgeWebView2Setup.exe`** installer, which you can integrate into your existing deployment process. The runtime will continue to be downloaded directly from the internet (as in the first scenario), but you won't need to download the bootstrapper as well during the setup.
- Next is using an offline installer. In this case, from the same **`WebView2`** runtime download page, you must choose one of the options under the **Evergreen Standalone Installer** section, based on the CPU architecture you're targeting. This installer is tailored for offline scenarios since the runtime won't be downloaded in real time from the internet, but it's fully included. As a downside, your installer will be bigger since, other than your application, you're going to include the full runtime (potentially in multiple versions, if you need to target different CPU architectures). The bootstrapper, instead, automatically takes care of downloading the correct version for the CPU architecture of the target machine. It's important to highlight, however, that, even if the first installation is done in offline mode, you will continue to benefit from the Evergreen distribution model: as long as your machine can connect to the internet, you will automatically get updates when they're available.

This distribution model is a good fit for applications that are deployed on machines that are connected to the internet.

Distributing the fixed version runtime

The fixed version runtime can be downloaded, as well, from the official **`WebView2`** Runtime download page at **https://developer.microsoft.com/en-us/microsoft-edge/webview2/#download-section**. You must choose a specific version and architecture: what you will get is a CAB file (with an approximate size of 160 MB) with the entire runtime.

In this scenario, there isn't a real installation, since the runtime won't be installed system-wide: it will be specific only for your application and it will never be automatically updated. You will need to embed the files that compose the runtime directly inside your package, along with the other binaries of your application. The first step, as such, is to unpack the CAB file (using a utility such as 7-Zip or WinRAR) inside your application's package. It can be the package itself or a subfolder. There isn't any specific rule; you can use the approach that works best for your needs.

If the evergreen version runtime doesn't require any specific configuration in the application (it will be picked up automatically), in the case of the fixed version you will instead need to make a few code changes, since we must instruct the **`WebView2`** control where to find the binaries. In a .NET application, you can do this by setting the **`BrowserExecutableFolder`** property of the **`CreationProperties`** object exposed by the **`WebView2`** control, as in the following example:

```
MyWebView.CreationProperties.BrowserExecutableFolder =
  $"{Environment.CurrentDirectory}\\WebView2Runtime";
```

In this code, we're assuming that the runtime will be stored in a folder called **`WebView2Runtime`**, which is included in the root folder where the application has been deployed. You must use this initialization code before setting the **`Source`** property in XAML or calling the **`EnsureCoreWebView2Async()`** method.

Thanks to this section, now we have all the tools we need to start building hybrid applications and to reuse our investments in the web ecosystem in our Windows applications.

Summary

In this chapter, we have explored many ways in which Windows isn't just a host for our applications, but can provide many features that

can really make the difference between a native application and a web one. We have learned how .NET makes it easy to integrate APIs from the UWP, something that in the past was available only if you wanted to start building a new application from scratch. The UWP unlocks endless opportunities, and, in this chapter, we have only touched on a few of them, such as location services, Windows Hello, the sharing contract, and the new filesystem capabilities.

Another thing we have learned in this chapter is that yes, it's true, this book is dedicated to building desktop applications but, as developers, we can't deny the impact of the web ecosystem. Therefore, Microsoft started from scratch with a new engine to build the new Microsoft Edge, which enables developers to get the best out of the web, thanks to the latest HTML, CSS, and JavaScript capabilities, not to mention the support for new features such as WebAssembly, WebVR, and so on. Through the **`WebView2`** control, included in the Windows App SDK, we can bring these features directly into our desktop applications and create hybrid experiences that can deliver the best of both worlds. In this chapter, we have learned about the features that make it possible, such as enabling communication between a web application and a desktop one.

We have just scratched the surface since the integration opportunities with the Windows ecosystem are endless. In the next chapter, we're going to focus on a powerful feature offered by Windows: the ability to integrate artificial intelligence directly into our applications, without using an online service.

Questions

1. To use the **`WebView2`** control in desktop applications, you just need to have the latest version of the Microsoft Edge browser installed on your machine. True or false?

2. To use UWP APIs in a .NET application you must use a library called C#/WinRT, which you must install from NuGet as the first step. True or false?
3. The new **`StorageFile`** and **`StorageFolder`** APIs can also be used in unpackaged applications. True or false?